SECURE RANDOM PASSWORD GENERATOR

Python Essentials – Build Your Own Project

Command-Line Python Application

Project Report

1. Introduction

This project is a command-line application that generates secure random passwords according to user-selected requirements. It also provides a basic password-strength assessment and an estimated entropy value. The application is designed using Python standard-library modules so that an evaluator can execute it directly from a terminal.

2. Problem Statement

Manually creating passwords can produce predictable or repeated passwords. A lightweight command-line tool can provide a convenient way to create random passwords while allowing users to control length and character categories.

3. Functional Requirements

FR1: Generate a password using secure randomness.
FR2: Allow a custom length from 8 to 128 characters.
FR3: Allow lowercase, uppercase, numbers, and special characters to be selected.
FR4: Generate multiple passwords in one execution.
FR5: Check password strength and estimate entropy.
FR6: Validate invalid input and display useful errors.
FR7: Support both interactive and command-line execution.
FR8: Provide automated tests.

4. Non-functional Requirements

Security: use the Python secrets module and do not store generated passwords.
Usability: provide clear prompts and command-line help.
Reliability: validate user inputs and handle common errors.
Maintainability: separate generation, validation, UI, CLI, configuration, and strength analysis into modules.
Performance: password generation completes immediately for normal requested sizes.
Resource efficiency: only standard-library modules are required.

5. System Architecture

The application has a thin entry point, an interactive UI layer, a CLI layer, validation utilities, a password-generation module, a strength-analysis module, and a configuration module. The generator uses secrets.choice and secrets.SystemRandom so that generated values are not based on the ordinary pseudo-random random module.

6. Design Diagrams

The repository contains architecture and workflow diagrams in the docs/ directory. The workflow is: receive requirements → validate input → build character pools → generate mandatory characters → fill remaining positions → securely shuffle → display password → calculate strength information.

7. Design Decisions & Rationale

The secrets module was selected because password generation is security-sensitive. A modular structure was selected to separate responsibilities and simplify testing. The application avoids persistent storage because storing generated passwords would create an unnecessary security risk. Standard-library-only execution reduces setup problems during automated evaluation.

8. Implementation Details

The generator guarantees at least one character from each enabled category when the requested length permits it. Remaining characters are selected from the combined pool and the result is securely shuffled. The strength checker examines length and character diversity and calculates an estimated entropy value from the detected character pool.

9. Screenshots / Results

Expected command-line operations include:

```
python main.py
python main.py generate --length 20 --count 3
python main.py check "ExamplePassword123!"
```

The README provides complete run instructions.

10. Testing Approach

Unit tests are implemented with Python's unittest framework. Tests cover password length, presence of selected character categories, strength analysis, entropy calculation, and invalid length handling. Run the suite using: python -m unittest discover -s tests -v

11. Challenges Faced

The main design challenge was balancing ease of use with secure generation. Another challenge was ensuring that every enabled character category is represented without creating predictable output. Input validation was separated from generation to keep error handling consistent.

12. Learnings & Key Takeaways

The project demonstrates Python package organization, functions, dictionaries, list processing, exception handling, command-line arguments, secure randomness, validation, and unit testing. It also demonstrates how a small script can be organized as a maintainable software project.

13. Future Enhancements

Possible enhancements include optional password policy presets, a passphrase mode, configurable character exclusions, stronger password auditing against common-password lists, and optional clipboard integration with an explicit user-controlled security warning.

14. References

Python documentation for the secrets module, argparse, unittest, and standard string handling. The project was designed for the VITyarthi Python Essentials Build Your Own Project requirements.

Appendix A – Project File Structure

File	Purpose
main.py	Application entry point
src/generator.py	Secure password generation
src/strength.py	Strength and entropy analysis
src/validators.py	Input validation
src/cli.py	Command-line argument handling
src/ui.py	Interactive terminal interface
src/config.py	Constants and character sets
tests/test_password_generator.py	Automated tests
README.md	Installation, usage, and testing instructions
statement.md	Problem statement and project scope